DeepJIT, PDL and determinism switches

Compiling DeepGEMM's kernels once the shape is known

DeepSeek-AI · DeepGEMM (deck 25) · DeepJIT

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Kernel Fusion & Runtime · github.com/deepseek-ai/DeepJIT

Every DeepGEMM kernel is compiled on first use

- Installing the library compiles no CUDA kernels
- `compiled_dims = "nk"`: N and K become constants, M stays dynamic
- The shapes arrive at run time: tokens, experts, context length

A power-of-two tile can idle 20 SMs

112 / 132

thread blocks that get an SM at M = 256, N = 7168 with a 128x128 tile — BLOCK_N = 112 fills 128 of 132 instead

DeepGEMM README @ commit 8002b769 (2025-03-25), section "Unaligned block sizes"

Ahead-of-time libraries pick tiles before the shape exists

- A template library freezes its tile set at build time
- **Autotuning** pays a benchmark at startup, and the winner varies per machine

DeepGEMM selects block sizes, warpgroups, stages and cluster size deterministically — "without auto-tuning"

Insight: compile at the first call

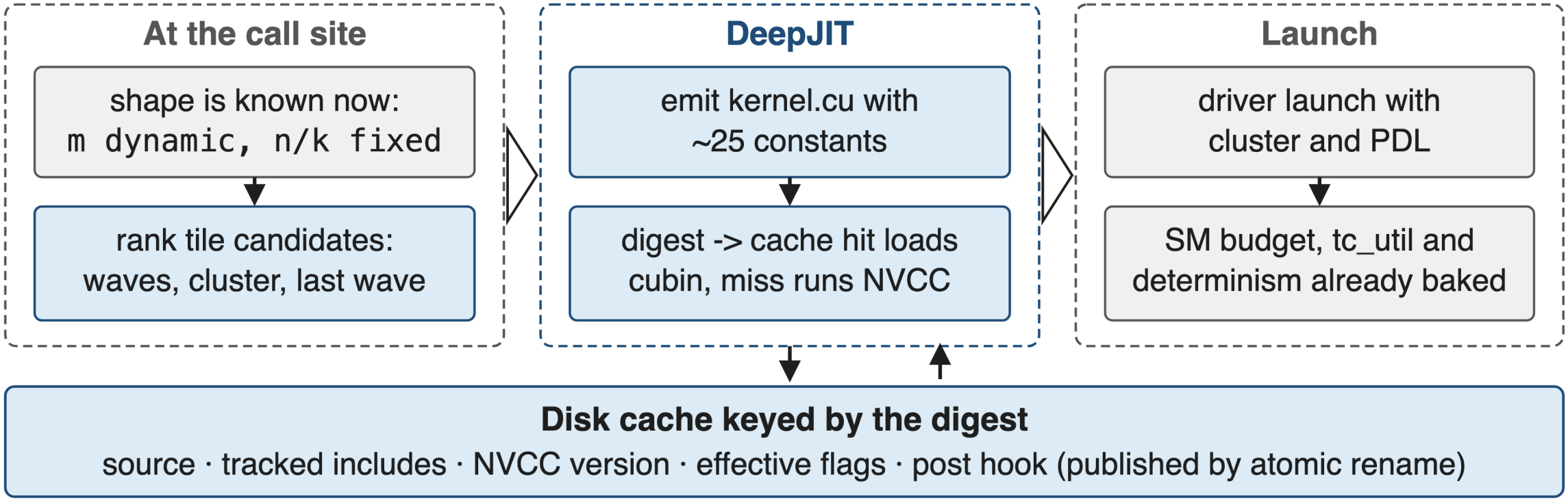
- Then the shape is a constant: scheduler bounds, stages, SM budget, switches
- The compiler can unroll the MMA pipeline fully and spend fewer registers

Insight: compile at the first call

- Then the shape is a constant: scheduler bounds, stages, SM budget, switches
- The compiler can unroll the MMA pipeline fully and spend fewer registers

One compile per shape and config, then a content-addressed cache that every process on the node shares

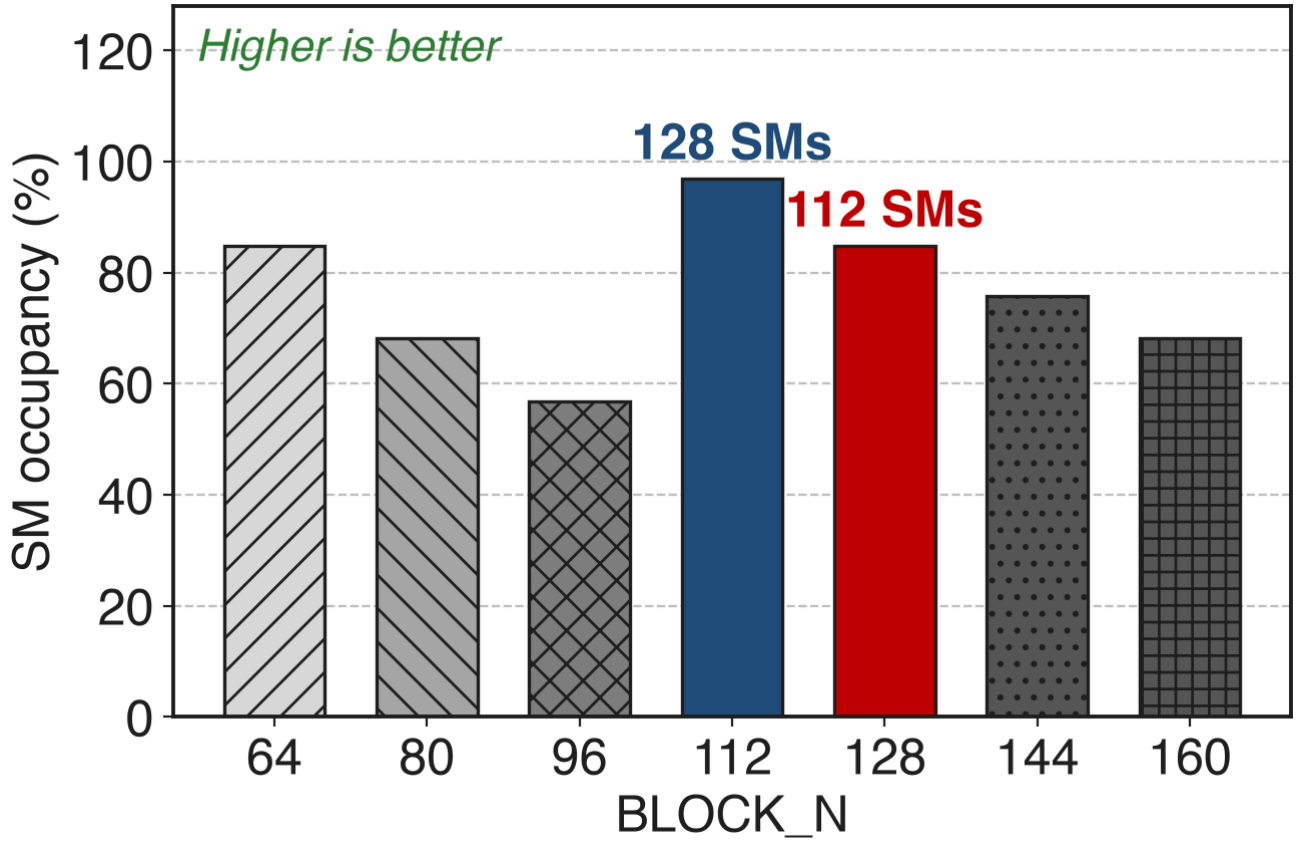
DeepJIT turns a call into a cached cubin



The digest covers the source, tracked includes, NVCC version, flags and post hook

❶ The heuristic ranks tiles, never benchmarks

- Candidates: swap-AB x BLOCK_M x BLOCK_N x cluster
- Ranked by single wave, multicast, fewer waves, last-wave fill



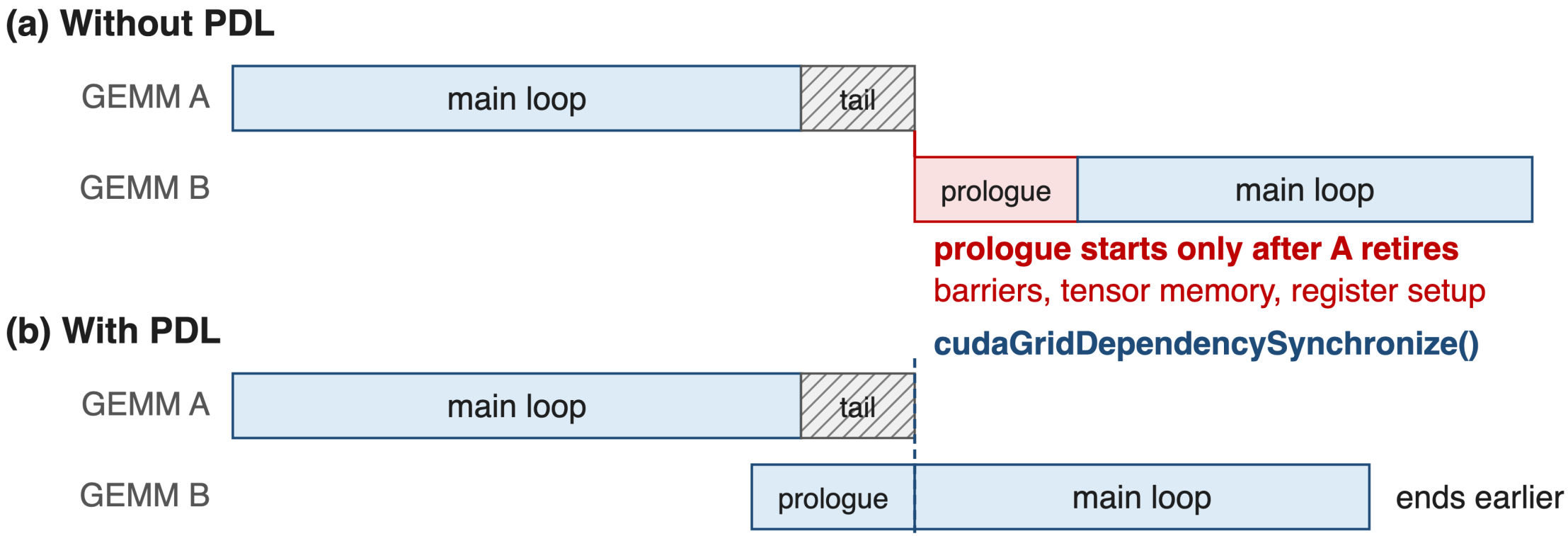
analytic: blocks that get one of the 132 SMs at M = 256, N = 7168

What the JIT burns into the generated source

- BLOCK_M/N/K, pipeline stages, swizzle modes, cluster size, thread counts
- `num_sms`, the tensor-core utilisation cap, the epilogue, the dtypes

`set_tc_util(80)` compiles in idle cycles after each MMA so the clock does not drop

❷ PDL overlaps the next kernel's prologue



Every kernel calls cudaGridDependencySynchronize() after its own setup work

❸ One switch buys bitwise batch invariance

- `use_deterministic_algorithms(True)`: no cuBLASLt dispatch, no split-K
- Mega mHC falls back to a fixed 16-way split, reduced in a later kernel
- The test suite re-runs each GEMM **20 times** and demands bitwise equality

DeepSeek-V4 replaced cuBLAS to keep tokens stable

- Goal: bitwise alignment across pre-training, post-training and inference
- Split-K abandoned in most scenarios, then recovered by other optimisations

cuBLAS "cannot achieve batch invariance", so V4 replaced it end-to-end with DeepGEMM (§3.3)

The cost: the first call compiles

- SGLang ships a precompile pass with a **3600 s** default timeout; vLLM a warm-up module
- Kernels must be compiled and loaded **before** CUDA Graph capture
- The digest tracks includes, NVCC version and flags; other deps need `extra_signature`

Summary

- A fixed tile can leave **20 of 132** SMs idle; the right tile fills 128
- **Insight**: compile at the first call, when the shape is still a constant
- Ranking without benchmarking, PDL overlap, SM and determinism switches baked in
- **Bitwise batch invariance** on demand; V4 replaced cuBLAS end-to-end

https://github.com/deepseek-ai/DeepJIT